

Lezione — Modulo 3A: Esecuzione e Monitoraggio dei Servizi

Corso: Lab Amministrazione di Sistemi T

Materiale: gestione_servizi.pdf (teoria) + LAB 31 marzo

Prerequisiti: Blocchi 0, 1, 2 completati

1. Demoni

Un **demone** (daemon) è un processo di sistema che: - viene avviato e arrestato automaticamente - non è collegato a nessun terminale - gira a nome di un utente specifico, senza shell interattiva

Le operazioni tipiche dell'amministratore sui demoni: - configurarne avvio e arresto automatico - verificarne lo stato - tracciarne l'esecuzione (log) - avviarli, arrestarli o inviare segnali manualmente

Perché è importante: i processi inutili consumano risorse e ampliano la superficie d'attacco. Ogni servizio attivo è una porta potenziale.

2. Le tre fonti primarie di processi

Oltre agli utenti interattivi, i processi su un sistema Linux vengono da tre fonti:

1. **Procedure di avvio del sistema** → gestite dal sistema di init
 2. **Pianificatori periodici e sporadici** → cron, at, timer systemd
 3. **Demoni di gestione degli eventi** → udev, D-Bus
-

3. Evoluzione del sistema di init

Il processo con PID 1 si chiama **init**. È il primo avviato dal kernel ed è responsabile (direttamente o indirettamente) di tutti gli altri processi.

3.1 SysVinit (storico)

- Configurato da `/etc/inittab`
- Gestisce **runlevel** (stati del sistema con sottoinsiemi di servizi attivi)
- Avvio **sequenziale**: lento, nessuna gestione errori
- Gli script dei servizi stanno in `/etc/init.d/`, symlinkati da `/etc/rcN.d/`
- Comandi: `/etc/init.d/nomeservizio {start|stop|status}`
- Su Debian: `update-rc.d nome enable/disable`

3.2 Upstart (Ubuntu 6.10–14.10, ora defunto)

- Basato su logica a **eventi** — avvio parallelo e non bloccante
- Ogni task risponde ad eventi (avvio hardware, avvio rete, ecc.)
- Comando: `initctl start/stop/status`

3.3 Systemd (attuale standard)

- Avvio **parallelo condizionato** — ogni unit parte appena i vincoli di dipendenza sono rispettati
 - Sostituisce init, udev, cron, syslog e molto altro
 - Usato da Debian 8+, Ubuntu 15.04+, praticamente tutto il mondo Linux moderno
-

4. Systemd — Concetti fondamentali

4.1 Unit

Systemd gestisce il sistema tramite **unit** — file di configurazione con nome nella forma `nome.tipo`.

Tipo	Funzione
<code>.service</code>	controllo e monitoraggio di un demone
<code>.socket</code>	attivazione via canale IPC (porta di rete, socket UNIX)
<code>.target</code>	gruppo di unit — sostituisce il concetto di runlevel
<code>.timer</code>	attività legate al tempo (sostituisce cron)
<code>.device</code>	punti di accesso a dispositivi hardware
<code>.mount</code> / <code>.automount</code>	punti di montaggio filesystem
<code>.slice</code>	gestione risorse via cgroup

4.2 Dove trovare le unit

<code>/lib/systemd/system/</code>	← libreria di riferimento (pacchetti software)
<code>/usr/lib/systemd/system/</code>	← stessa cosa su alcune distro
<code>/etc/systemd/system/</code>	← personalizzazioni locali (priorità massima)

Le personalizzazioni in `/etc/systemd/system/` sovrascrivono quelle di sistema.

5. systemctl — Controllo dei servizi

5.1 Operazioni a runtime (volatili — nulla cambia alla configurazione)

```
systemctl start  nomeservizio  # avvia
systemctl stop   nomeservizio  # ferma
systemctl restart nomeservizio  # stop + start
systemctl reload nomeservizio  # ricarica configurazione senza restart
systemctl status nomeservizio  # mostra stato, PID, log recenti
```

`systemctl status` mostra: stato corrente, process tree, risorse (CPU, cgroup), ultimi log da journald.

5.2 Configurazione persistente (effetto al prossimo boot)

```
systemctl enable  nomeservizio  # avvio automatico al boot
systemctl disable nomeservizio  # disabilita avvio automatico (ma manuale rimane possibile)
```

```
systemctl mask    nomeservizio    # neutralizza completamente (impedisce anche start manuale)
systemctl unmask  nomeservizio    # ripristina
```

`disable mask`: `disable` rimuove l'avvio automatico ma lasci fare **start** manualmente.
`mask` lo rende completamente inerte.

5.3 Ispezione della configurazione

```
systemctl list-units                # tutte le unit attive
systemctl list-units -t service    # solo i servizi attivi
systemctl list-unit-files -t service # tutti i servizi installati (anche inattivi)
systemctl list-unit-files -t service --state=enabled # solo quelli abilitati al boot
systemctl --state=failed           # unit in stato failed
```

5.4 Target — i nuovi runlevel

SysV Runlevel	Systemd Target	Note
0	<code>poweroff.target</code>	Spegnimento
1	<code>rescue.target</code>	Single user mode
3	<code>multi-user.target</code>	Multi-utente, senza GUI
5	<code>graphical.target</code>	Multi-utente con GUI
6	<code>reboot.target</code>	Riavvio

```
systemctl get-default                # target di avvio corrente
systemctl set-default multi-user.target # imposta target di default
```

5.5 Contesto utente

Di default `systemctl` opera sul service manager di **sistema** (richiede root). Ma ogni utente ha il proprio service manager, attivabile con:

```
systemctl --user start  mioservizio
systemctl --user enable mioservizio    # avvio automatico al login (non al boot)
```

6. Struttura di un unit file

Un unit file `.service` minimo:

```
[Unit]
Description=Il mio servizio di esempio
Requires=network.target
After=network.target

[Service]
Type=exec
ExecStart=/usr/bin/mioprogramma --opzione
ExecStop=/usr/bin/mioprogramma --stop
Restart=on-failure
```

```
[Install]
WantedBy=multi-user.target
```

6.1 Sezione [Unit] — dipendenze

Direttiva	Significato
Requires=B	B deve essere attiva; se B fallisce, questa si ferma
Wants=B	come Requires ma tollerante: se B fallisce, questa continua
After=B	avvia <i>dopo</i> B (Wants/Requires non implicano l'ordine temporale!)
Conflicts=B	mutuamente esclusivi con B
OnFailure=C	avvia C se questa fallisce
WantedBy= (in [Install])	crea automaticamente un Wants nella unit indicata all'enable

Regola importante: Wants=B senza After=B significa “avvia in parallelo con B e verifica dopo”. Per avvio in sequenza servono entrambi.

6.2 Tipi di avvio (Type= nella sezione [Service])

Type	Comportamento
exec	systemd considera il servizio avviato quando il comando ExecStart è effettivamente in esecuzione (<i>consigliato per i nuovi servizi</i>)
simple	come exec ma systemd dichiara il servizio avviato già al fork — meno sicuro
forking	il processo si sdoppia e il genitore esce (demoni Unix classici)
notify	come exec, ma il programma invia esplicitamente segnale di readiness via <code>sd_notify</code>
oneshot	esegue una volta sola e termina; utile per script di configurazione
idle	attende che tutti gli altri job siano smistati, poi parte

Per servizi che configurano qualcosa all'avvio e la ripristinano allo stop (tipico **oneshot**):

```
ExecStart=/usr/bin/mio-configuratore
ExecStop=/usr/bin/mio-de-configuratore
```

6.3 Riavvio automatico (Restart=)

```
Restart=on-failure    # riavvia se termina con exit code non zero o segnale non clean
Restart=always        # riavvia sempre, tranne se fermato da systemctl stop
```

7. Esecuzioni pianificate

7.1 at — esecuzione differita (una tantum)

at pianifica un comando da eseguire una volta sola in un momento futuro.

```
sudo apt install at          # installa se non presente
```

```
# Pianificare
echo "/bin/comando" | at now + 30 minutes
echo "/bin/comando" | at 14:30
echo "/bin/comando" | at midnight 31.12.2025
```

```
# Gestire la coda
atq          # lista job in coda
atrm <job_id> # rimuove un job
```

Pattern per salvare l'ID del job (per poterlo cancellare):

```
ATJOB=$(mktemp)
echo "/bin/comando" | at now + 30 minutes 2>&1 | grep '^job ' | cut -f2 -d' ' > $ATJOB
# Per cancellare:
atrm $(cat $ATJOB)
```

7.2 cron — esecuzione periodica

crond legge i file crontab ogni minuto e lancia i task pianificati.

```
crontab -l          # mostra il crontab dell'utente corrente
crontab -e          # edita il crontab
crontab FILE        # sostituisce il crontab con FILE
```

Sintassi crontab:

```
MINUTO  ORA  G.MESE  MESE  G.SETTIMANA  <comando>
```

Esempi:

```
30 4 1,15 * 6      /bin/backup    # alle 4:30 del giorno 1, 15 e ogni sabato
*/10 8-18 * * 1-5  /home/las/lavora.sh # ogni 10 min, 8-18h, lun-ven
0 0 1 1 *          /usr/sbin/auguri    # 1 gennaio a mezzanotte
```

Eccezione: se sia G.MESE che G.SETTIMANA sono specificati (entrambi *), vengono considerati in **OR logico**, non AND.

Crontab di sistema (/etc/crontab) ha un campo aggiuntivo per specificare l'utente:

```
30 4 * * * root /usr/sbin/backup
```

Manipolazione programmatica del crontab (da script):

```
TMPTAB=$(mktemp)
crontab -l | grep -vF "$COMANDO" > $TMPTAB  # dump senza duplicati
echo "$PERIODO $COMANDO" >> $TMPTAB
crontab $TMPTAB
rm $TMPTAB
```

7.3 Timer systemd — sostituzione moderna di cron e at

Timer periodico (equivalente cron):

Creare due file: il servizio da eseguire e il timer che lo attiva.

/etc/systemd/system/myscript.service:

```
[Unit]
Description=Il mio script periodico
```

```
[Service]
Type=oneshot
ExecStart=/path/to/script.sh
```

/etc/systemd/system/myscript.timer:

```
[Unit]
Description=Timer per myscript

[Timer]
OnCalendar=*-*-* 04:30:00    # ogni giorno alle 4:30
Persistent=true             # esegue anche se la macchina era spenta
```

```
[Install]
WantedBy=timers.target

sudo systemctl enable --now myscript.timer    # attiva il timer subito e al boot
systemctl list-timers                          # mostra tutti i timer attivi
```

Timer differito (equivalente at) — usa systemd-run:

```
systemd-run --on-active=30m /path/to/script.sh          # tra 30 minuti
systemd-run --on-calendar="2025-12-24 11:00:00" /path/to/script.sh
systemd-run --user --on-active=1m /bin/cmd               # come utente normale
```

Sintassi OnCalendar:

DayOfWeek YYYY-MM-DD HH:MM:SS

- * = qualsiasi valore
- A..B = range
- A,B,C = lista
- /P = incrementi di P unità (es. */2 = ogni 2)

Verificare la prossima scadenza di un'espressione: `systemd-analyze calendar "Mon *-*-* 04:00"`

8. Monitoraggio

8.1 Log di sistema

I log sono indispensabili per diagnostica e rilevamento attività malevole. Vanno protetti (integrity checker, replica su macchine remote).

Architettura syslog — principi mantenuti da tutte le implementazioni: 1. I processi inviano messaggi al logger 2. Il logger serializza, fa timestamping e classifica 3. Il logger smista verso destinazioni configurate

Ogni messaggio ha una coppia <facility>.<priority>: - **Facility**: argomento del messaggio (auth, cron, daemon, kern, mail, user, local0–local7, ...) - **Priority** (decrescente): emerg, alert, crit, err, warning, notice, info, debug

Una regola specifica una priority come **soglia**: fa match con quella priority e tutte le superiori, a meno che non sia preceduta da = (exact match).

8.2 rsyslog (standard su Debian)

rsyslog è il successore di syslog, installato di default su Debian.

File di configurazione: - Direttive globali: /etc/rsyslog.conf - Direttive aggiuntive: /etc/rsyslog.d/*.conf ← qui si aggiungono le proprie regole

Sintassi di una regola:

facility.priority /path/al/file/di/destinazione

Esempi:

kern.*	/dev/console	# tutto del kernel → console
*.info;mail.none	/var/log/messages	# tutto info+ tranne mail
*.emerg	*	# emergenze → tutti gli utenti
kern.crit	/usr/bin/alerter	# critici del kernel → programma
*.=warning	@loghost	# solo warning → server remoto UDP

Aggiungere una facility personalizzata:

```
echo "local4.=info /var/log/myevents.log" > /etc/rsyslog.d/mylog.conf
systemctl restart rsyslog
# Per disabilitarla:
mv /etc/rsyslog.d/mylog.conf /etc/rsyslog.d/mylog.off
systemctl restart rsyslog
```

8.3 systemd-journald

journald è il sistema di log integrato in systemd: - formato **binario** (non testo) → visualizzabile solo con journalctl - attivo dal boot, non dipende da altri servizi - rotazione automatica per età o dimensione

journalctl	# tutto il journal
journalctl -u nomeservizio	# log di un servizio specifico
journalctl -f	# segui in tempo reale (come tail -f)
journalctl --since "1 hour ago"	# filtra per tempo
journalctl -p err	# solo priority err e superiori
journalctl -b	# log del boot corrente
journalctl -b -1	# log del boot precedente

journald + rsyslog insieme: aggiungendo `ForwardToSyslog=yes` in `/etc/systemd/journald.conf`, i messaggi raccolti da `journald` vengono inoltrati a `rsyslog` (che può poi mandarli a server remoto o separarli in file di testo).

8.4 Diagnostica istantanea del sistema

Comando	Cosa mostra
<code>ps aux</code>	snapshot di tutti i processi attivi
<code>uptime</code>	tempo di attività + load average (1', 5', 15')
<code>free</code>	occupazione RAM e swap
<code>top</code>	combinazione interattiva di <code>ps+uptime+free+CPU</code>
<code>vmstat 1</code>	memoria, paging, I/O, CPU aggiornato ogni secondo
<code>iostat /dev/sda1</code>	statistiche I/O su un dispositivo
<code>df</code>	spazio disco per filesystem
<code>du -s /directory</code>	spazio occupato da una directory
<code>fuser /path</code>	quali processi usano un file/filesystem
<code>lsuf</code>	tutti i file aperti da tutti i processi

Lettura di `uptime`:

```
21:27:56 up 7:10, 2 users, load average: 0.00, 0.00, 0.00
      tempo attivo   utenti    1'    5'   15'
```

Il load average va interpretato in rapporto al numero di CPU: load=2 su 4 CPU = sistema a metà carico.

Lettura di `free`:

	total	used	free	shared	buff/cache	available
Mem:	237040	121248	9596	1464	106196	98720

available free + buff/cache — la cache può essere liberata se serve.

systemctl status come strumento di diagnostica — mostra in un unico output: configurazione unit, stato corrente, PID, albero processi, uso risorse (CPU, cgroup), ultimi messaggi di log da `journald`.

9. Esercizi del Lab (31 marzo)

Es. 1 — Installare `at` e pianificare un comando

```
sudo apt install at
```

```
# Pianifica un comando tra 30 minuti
echo "/bin/echo ciao" | at now + 30 minutes
```

```
# Verifica la coda
atq
```

```
# Cancella il job (sostituire N con il numero del job)
atrm N
```

Es. 2 — Watchdog con at

Pattern: limita il tempo di esecuzione di un processo.

```
COMANDO_LUNGO &
PID=$!
WD=$(mktemp)
echo "/bin/kill $PID" | at now + $LIMITE minutes 2>&1 | grep '^job ' | cut -f1 -d' ' > $WD
# Al completamento normale: atrm $(cat $WD)
```

Es. 3 — Nice execution (script con at)

Logica: se load < soglia, esegui il comando; altrimenti rischedula tra 2 minuti con at.

```
THIS=/home/vagrant/niceexec-base.sh
LOAD=$(uptime | awk -F 'average: ' '{ print $2 }' | cut -f1 -d$(locale decimal_point))
if test $LOAD -lt "$1" ; then
    shift
    eval "$@"
else
    echo $THIS "$@" | at now +2 minutes
fi
```

Es. 4 — Manipolazione crontab da script

```
PERIODO='*/10 8-18 * * *'
COMANDO='/home/las/qualcosa.sh parametri vari'
TMPTAB=$(mktemp)
crontab -l | grep -vF "$COMANDO" > $TMPTAB
echo "$PERIODO $COMANDO" >> $TMPTAB
crontab $TMPTAB
rm $TMPTAB
```

Es. 5 — Configurazione rsyslog

```
# Aggiunge una facility
echo "local4.=info /var/log/myevents.log" | sudo tee /etc/rsyslog.d/mylog.conf
sudo systemctl restart rsyslog

# Testa con logger
logger -p local4.info "messaggio di test"
tail /var/log/myevents.log
```

Es. 6 — Esercizio logconfig

Script che prende facility.priority e un percorso file, e: 1. Configura rsyslog per loggare quella facility sul file dato 2. Configura cron per mandare ogni 5 minuti l'output di uptime a syslog con quella etichetta

Soluzione (schema essenziale):

```
#!/bin/bash
# Validazioni: $1 deve essere facility.priority, $2 un path assoluto
T=$(mktemp)
echo -e "$1\t$2" > $T
sudo cp $T /etc/rsyslog.d/logconfig.conf
sudo systemctl restart rsyslog.service

CRONCOMMAND="/usr/bin/uptime | /usr/bin/logger -p $1"
crontab -l | grep -Fv "$CRONCOMMAND" > $T
echo "*/* * * * * $CRONCOMMAND" >> $T
crontab $T
rm -f $T
```

Es. 7 — Esaminare unit systemd reali

```
# Unit oneshot (configura qualcosa e poi ripristina)
cat /lib/systemd/system/networking.service

# Unit forking (demone in background)
cat /lib/systemd/system/dnsmasq.service

# Unit con Restart (riavvio automatico)
cat /lib/systemd/system/cron.service
cat /lib/systemd/system/rsyslog.service

# Controllare log di una unit
journalctl -u ssh.service
journalctl -u cron.service -f
```

10. Cheat sheet systemctl

Operazione	Comando
Avvia ora	<code>systemctl start nome</code>
Ferma ora	<code>systemctl stop nome</code>
Stato + log	<code>systemctl status nome</code>
Riavvia	<code>systemctl restart nome</code>
Ricarica config	<code>systemctl reload nome</code>
Abilita al boot	<code>systemctl enable nome</code>
Disabilita al boot	<code>systemctl disable nome</code>
Blocca completamente	<code>systemctl mask nome</code>
Lista servizi attivi	<code>systemctl list-units -t service</code>
Lista tutti i servizi	<code>systemctl list-unit-files -t service</code>
Solo i falliti	<code>systemctl --state=failed</code>
Target corrente	<code>systemctl get-default</code>

Operazione	Comando
Log di un servizio	<code>journalctl -u nome</code>
Log in tempo reale	<code>journalctl -u nome -f</code>

Collegati

- [[Appunti_modulo3A]]
- [[appunti_modulo3A_systemd_servizi]]

Hub: [[master_map_studio]] · [[glossario_sysadm]] · [[concept_maps]] · [[troubleshooting_vm]]
 · [[metodo_studio_esami_pratici]]